

1. Cover Page

Cyber security assessment(Secure Corp)

Name- P.Gokkul Krishnaa

Role- Junior cyber security analyst

Date- 01-09-2026

2. Executive Summary

1. This project was a security assessment using

Wsl(attacker)

Metasploitable2(DVWA)

Wazuh.(To see logs)

2. Nmap was used to identify open ports and services running on the target system.

3. Different attacks were tested, including brute force, SQL injection, XSS, command injection, FTP and SSH attacks.

4. Additional DVWA tests such as file upload, CSRF, file inclusion and blind SQL injection were also performed.

5. Screenshots and log evidence were collected during this process.

6. Wazuh was used for security log monitoring, and recommendations were prepared based on the findings.

3. Scope & Rules of Engagement

1. The assessment was performed only on the controlled lab environment created for this project.

2. Metasploitable2 and DVWA were used as the main targets for security testing.

3. The testing included network scanning, web application testing and controlled authentication attacks.

4. Only the services and applications available in the lab were tested.

5. The tests were performed for learning and security assessment purposes without targeting external systems.

6. Screenshots, command outputs and relevant logs were collected as evidence for the final report.

4. Asset Inventory & CIA Analysis

1. Metasploitable2, DVWA and Wazuh are the assets used in this lab.
2. Metasploitable2 is an intentionally vulnerable application, we tried some attacks on it as a part of testing.
3. The attacks were actually done on DVWA, which is running inside metasploitable2.
4. Wazuh is used for collecting logs(security related) from the lab space.
5. I performed attacks during the assignment which violates CIA-triad(CONFIDENTIALITY-INTEGRITY-AVAILABILITY).
6. These vulnerabilities will become a problem when exposed to the real world.

Asset	Type	Owner / Department	Why It Matters
Metasploitable2	Hardware/Software*	Security / IT	Vulnerable lab system used for security testing
DVWA	Software	Security / IT	Used for web vulnerability testing
Wazuh	Software	Security / IT	Used for security log monitoring
Testing Machine	Hardware/Software*	Security Team	Used to perform security assessment
Network Connection	Connectivity	IT / Network Team	Allows communication between systems

Asset	Confidentiality	Integrity	Availability
DVWA	Prevent unauthorized access to application data	Prevent unauthorized changes	Application should remain accessible for users
Metasploitable2	Protect information stored on the system	Prevent unauthorized changes	System and services should remain available
Wazuh	Protect security logs	Logs should not be modified	Monitoring should remain available
Testing Machine	Protect assessment data	Prevent unauthorized changes	Available for security testing
Network Connection	Prevent unauthorized network access	Data should not be modified during transfer	Network communication should remain available

5. Risk Assessment

1. I tested some vulnerabilities which are intentional in it and reviewed according to their damage impact on the application.
2. Sql injection- Exposure of user's data in database.
3. XSS- Risk of harmful scripts that can affect user's data.
4. Command injection- Risk of running unwanted commands.
5. File inclusion- Risk of accessing unwanted files.
6. File upload- Risk of uploading unwanted files.
7. Brute force/SSH/FTP- Risk of unauthorized login.

Asset	Threat	Vulnerability	Impact	Risk Rating	Recommendation
DVWA	SQL Injection	Poor input validation	Database information may be exposed	High	Use input validation and parameterized queries
DVWA	XSS	Improper input handling	Harmful scripts may affect users	Medium	Sanitize input and encode output
DVWA	Command Injection	Unsafe command handling	Unauthorized commands may be executed	High	Validate input and avoid direct command execution
DVWA	File Inclusion	Unsafe file path handling	Private system files may be accessed	High	Restrict file paths and validate input
DVWA	File Upload	Weak file upload validation	Unwanted files may be uploaded	Medium	Validate file type, size and extension
DVWA	Brute Force	Weak login protection	Unauthorized login may occur	High	Use strong passwords and login rate limiting
Metasploitable2	FTP Brute Force	Weak FTP authentication	Unauthorized FTP access may occur	High	Use strong passwords and limit login attempts
Metasploitable2	SSH Brute Force	Weak SSH authentication	Unauthorized system access may occur	High	Use strong authentication and rate limiting

6. Methodology

- 1. I first installed WSL(ubuntu) for the attacking part and installed metasploitable2 folder , added it to the virtual box.**
- 2.Added ubuntu to the virtual box for accessing logs by adding Wazuh into it for logs information.**
- 3. I failed in connecting them, then I approached my mentor for the issue, he told me to use metasploitable2 docker image for the assignment.**
- 4.finally, i made the connection between them and created a lab environment.**
- 5. Performed attacks on DVWA,metasploitable2 as instructed.**
- 6.**
- 7.**

7. Attack Simulation Findings

Nmap Reconnaissance

I used Nmap to check the target system and find the open ports and running services.

The scan was performed using the following command:

```
nmap -Pn -sV -p 8080 localhost
```

The scan showed port 8080 open with Apache HTTP server running.

This helped me identify the web service before starting the security testing.

- 1. I opened the DVWA Brute Force page and set the security level to Low. I tried multiple username and password combinations using a small list. After a few attempts, the correct login details worked and the application showed the successful login message. I saved a screenshot as evidence.**

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More info

2. I opened the SQL Injection section in DVWA and kept the security level at Low. I entered the payload ``1' OR '1'='1`` in the input field. The application returned multiple user records instead of showing only one result. This showed that the

input was not properly handled and could be used to access database information. I saved a screenshot of the payload and the result as evidence.



Home

Instructions

Setup

Brute Force

Command Execution

CSRF

File Inclusion

SQL Injection

SQL Injection (Blind)

Upload

XSS reflected

XSS stored

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

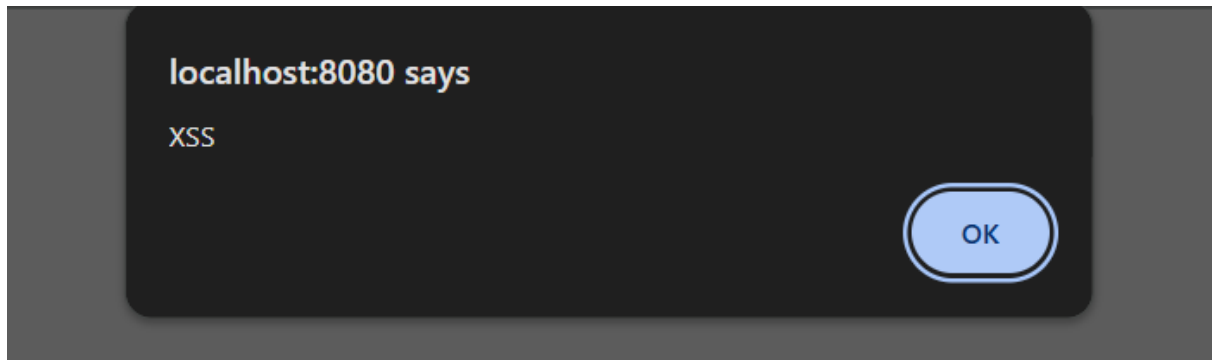
More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

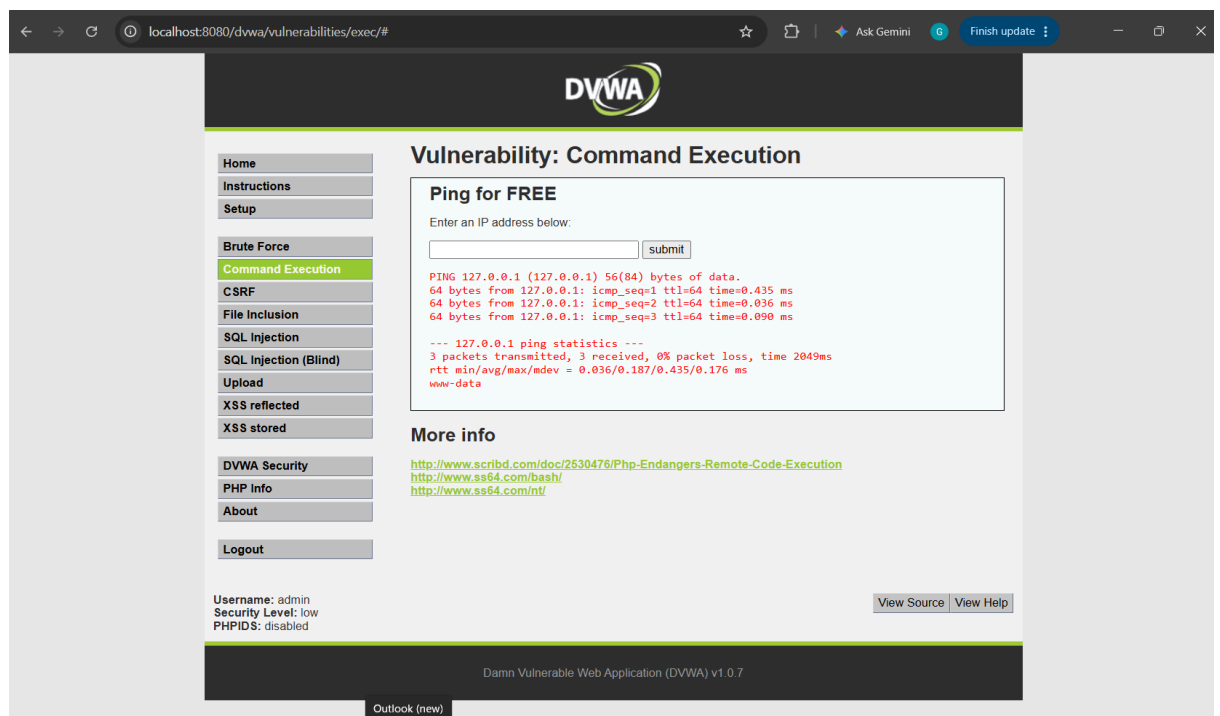
Username: admin
Security Level: low
PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.0.7

3. I opened the XSS section in DVWA and kept the security level at Low. I entered a simple script in the input field. The script was executed and an alert box appeared on the screen. This showed that the application was not properly filtering the input. I saved a screenshot of the successful result as evidence.



4. I opened the Command Execution section in DVWA and kept the security level at Low. I entered [127.0.0.1; whoami] in the input field. The application returned the result of the [whoami] command and showed [www-data]. This confirmed that an additional system command could be executed through the application. I saved a screenshot of the result as evidence.



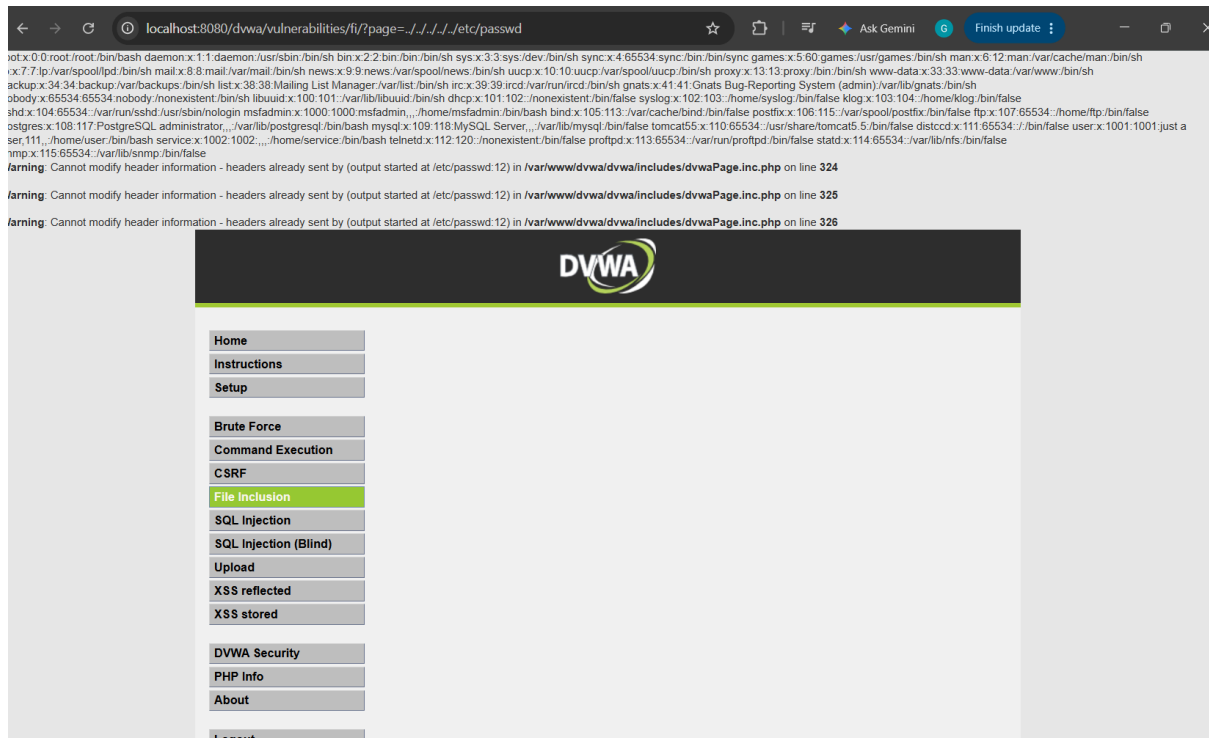
6. File inclusion-

I tested File Inclusion in DVWA with the security level set to Low.

I used the payload `../../../../../../etc/passwd` to access a local file.

The application displayed entries from the `/etc/passwd` file.

This showed that the application could be used to access local files without proper validation.



6. I connected to the FTP service running on Metasploitable2 and tried multiple login attempts with a small password list. Some login attempts failed and one login was successful. The FTP log recorded the failed and successful login attempts. I saved the FTP output/log screenshot as evidence.

```

PS C:\Users\Gokul> docker exec metasploitable2 sh -c "tail -n 20 /var/log/vsftpd.log"
Sun May 20 15:12:37 2012 [pid 5767] CONNECT: Client "127.0.0.1"
Sun May 20 15:12:39 2012 [pid 5766] [metasploitable2] FAIL LOGIN: Client "127.0.0.1"
Sun May 20 15:12:46 2012 [pid 5770] CONNECT: Client "127.0.0.1"
Sun May 20 15:12:46 2012 [pid 5769] [msfadmin] OK LOGIN: Client "127.0.0.1"
Sun May 20 15:13:10 2012 [pid 5784] CONNECT: Client "127.0.0.1"
Sun May 20 15:13:10 2012 [pid 5783] [msfadmin] OK LOGIN: Client "127.0.0.1"
Sun May 20 15:44:14 2012 [pid 5303] CONNECT: Client "127.0.0.1"
Sun May 20 15:48:53 2012 [pid 5278] CONNECT: Client "172.16.123.1"
Sat Sep 12 07:22:56 2026 [pid 48] CONNECT: Client "172.17.0.1"
Sat Sep 12 07:22:56 2026 [pid 50] CONNECT: Client "172.17.0.1"
Sat Sep 12 07:25:10 2026 [pid 52] CONNECT: Client "172.17.0.1"
Sat Sep 12 07:25:56 2026 [pid 51] [msfadmin] FAIL LOGIN: Client "172.17.0.1"
Sat Sep 12 07:28:34 2026 [pid 69] CONNECT: Client "172.17.0.1"
Sat Sep 12 07:28:45 2026 [pid 68] [msfadmin] OK LOGIN: Client "172.17.0.1"
Sat Sep 12 07:29:34 2026 [pid 72] CONNECT: Client "172.17.0.1"
Sat Sep 12 07:30:31 2026 [pid 71] [msfadmin] FAIL LOGIN: Client "172.17.0.1"
Sat Sep 12 07:30:49 2026 [pid 74] CONNECT: Client "172.17.0.1"
Sat Sep 12 07:30:58 2026 [pid 73] [msfadmin] FAIL LOGIN: Client "172.17.0.1"
Sat Sep 12 07:31:05 2026 [pid 76] CONNECT: Client "172.17.0.1"
Sat Sep 12 07:31:15 2026 [pid 75] [msfadmin] FAIL LOGIN: Client "172.17.0.1"
PS C:\Users\Gokul>

```

7. I connected to the SSH service running on Metasploitable2 and tested multiple login attempts using a small password list. The failed login attempts were recorded in the SSH authentication logs. I was also able to successfully connect to the SSH service using the correct login details. The SSH login activity was saved as evidence for further investigation in Wazuh.

```

gokul@LAPTOP-44D1E3NT: $ nmap -Pn -sV -p 2222 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-13 06:01 +0000
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000060s latency).

PORT      STATE SERVICE      VERSION
2222/tcp  open  tcpwrapped

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.41 seconds
gokul@LAPTOP-44D1E3NT: $ ssh -p 2222 msfadmin@127.0.0.1
key_exchange_identification: read: Connection reset by peer
Connection reset by 127.0.0.1 port 2222
gokul@LAPTOP-44D1E3NT: $ ssh -p 2222 msfadmin@127.0.0.1
key_exchange_identification: read: Connection reset by peer
Connection reset by 127.0.0.1 port 2222
gokul@LAPTOP-44D1E3NT: $ ssh -p 2222 msfadmin@127.0.0.1
Unable to negotiate with 127.0.0.1 port 2222: no matching host key type found. Their offer: ssh-rsa,ssh-dss

```

```

gokul@LAPTOP-44D1E3NT: $ ssh -p 2222 msfadmin@127.0.0.1
kex_exchange_identification: read: Connection reset by peer
Connection reset by 127.0.0.1 port 2222
gokul@LAPTOP-44D1E3NT: $ ssh -p 2222 msfadmin@127.0.0.1
kex_exchange_identification: read: Connection reset by peer
Connection reset by 127.0.0.1 port 2222
gokul@LAPTOP-44D1E3NT: $ ssh -p 2222 msfadmin@127.0.0.1
Unable to negotiate with 127.0.0.1 port 2222: no matching host key type found. Their offer: ssh-rsa,ssh-dss
gokul@LAPTOP-44D1E3NT: $ ssh -p 2222 -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@127.0.0.1
The authenticity of host '[127.0.0.1]:2222 ([127.0.0.1]:2222)' can't be established.
RSA key fingerprint is: SHA256:BQHm5EoHX9GCiOLuVscegPXLQ0suPs+E9d/rrJB84rk
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[127.0.0.1]:2222' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@127.0.0.1's password:
Linux 32554753bfe5 4.13.0-21-generic #24-Ubuntu SMP Mon Dec 18 17:29:16 UTC 2017 x86_64

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sun Jul 16 21:04:01 2017
msfadmin@8e6b3e02813d:~$ █

```

Wazuh Dashboard

Wazuh was used to monitor the security events generated during the testing. The dashboard was checked for recent events and logs related to the activities performed on the lab systems. The detection status of each attack was recorded based on the evidence available in Wazuh.

Recommendations-

- 1. DVWA Brute Force - Strong passwords, login attempt limit, Block if many input requests in less time(unreal).**
- 2. SQL Injection - Input validation, protection of user's data.**
- 3. XSS - Input filtering, output encoding.**
- 4. Command Injection - Input validation, safe command handling**
- 5. File Inclusion - Restrict file paths, input validation.**
- 6. File Upload - File type, extension and size validation.**
- 7. FTP Brute Force - Strong passwords, limit login attempts**
- 8. SSH Brute Force - Strong authentication, login attempt limit**

Area	Current State	Recommendation	Priority
DVWA Brute Force	Multiple login attempts were allowed	Add login attempt limits and use strong passwords	High
SQL Injection	Input was accepted without proper checking	Use input validation and parameterized queries	High
XSS	Script input was accepted and executed	Filter user input and encode output	High
Command Injection	User input allowed an extra command to run	Validate input and avoid unsafe command execution	High
FTP Brute Force	Multiple FTP login attempts were possible	Use strong passwords and limit failed attempts	High
SSH Brute Force	Multiple SSH login attempts were possible	Use strong authentication and limit login attempts	High

